

Graphisme en deux dimensions

Pour représenter graphiquement des données en deux dimensions, il faut prioritairement préparer la liste des abscisses et la liste des ordonnées correspondante. Pour cela, on peut utiliser les tableaux du module *numpy*.

1 Le module numpy

numpy (*numerical python*) est un module *Python* optimisé pour effectuer des calculs mathématiques de manière rapide et plus performante sur des structures comme les vecteurs, les matrices, ...

On commence par importer le module et lui donner l'alias *np* afin de faciliter son utilisation :

```
[1]: import numpy as np
```

1.1 Les fonctions usuelles

Dans le module *numpy* on en trouve plusieurs comme :

- Fonctions trigonométriques : *np.cos*, *np.sin*, ...
- Fonctions logarithmiques et exponentielles :
 - *np.log* pour le logarithme naturel;
 - *np.log2* et *np.log10* pour le logarithme à base 2 et à base 10, ...
 - *np.exp* pour l'exponentielle, ...
- La fonction *np.sqrt* qui renvoie la racine carrée et *np.absolute* pour la valeur absolue, ...

1.2 Tableau unidimensionnel (vecteur)

array est le type de structure de données utilisé pour représenter un tableau multidimensionnel et homogène d'éléments (même type) comme un vecteur (une dimension) une matrice (deux dimensions), ...

Pour définir un tableau unidimensionnel (vecteur) on peut utiliser le mot clé *array* suivi par une liste des coordonnées du vecteur.

Exemple

```
[2]: vecteur = np.array([1, 3, 7])    #crée un vecteur de coordonnées (1, 3, 7)
```

On peut même générer rapidement différents vecteurs en utilisant :

- `np.arange(début, fin, pas)` qui renvoie un vecteur ligne dont les valeurs sont en progression arithmétique de raison *pas*, de premier terme *début* et ne dépassant pas *fin*.
- `np.linspace(début, fin, nombre_élément)` qui génère un vecteur de nombres uniformément espacés sur l'intervalle *[début, fin]*.
- `np.ones` qui renvoie un vecteur rempli de 1.
- `np.zeros` qui renvoie un vecteur rempli de 0.

Exemples

```
[3]: v1 = np.arange(1, 10, 0.5)    #un vecteur de valeurs d'une suite arithmétique de
    ↪raison 0.5 à partir de 1 et < 10

print(v1)
```

```
[1.  1.5 2.   2.5 3.   3.5 4.   4.5 5.   5.5 6.   6.5 7.   7.5 8.   8.5 9.   9.5]
```

```
[4]: v2 = np.linspace(1, 10, 20)   #un vecteur de 20 valeurs uniformément
    ↪distribuées sur [1, 10]

print(v2)
```

```
[ 1.          1.47368421  1.94736842  2.42105263  2.89473684  3.36842105
  3.84210526  4.31578947  4.78947368  5.26315789  5.73684211  6.21052632
  6.68421053  7.15789474  7.63157895  8.10526316  8.57894737  9.05263158
  9.52631579 10.          ]
```

Généralement, *arange* est utilisée lorsqu'on connaît le pas et *linspace* lorsqu'on connaît le nombre de valeurs.

```
[5]: v3 = np.ones(7)    #un vecteur contenant sept 'un'

print(v3)
```

```
[1.  1.  1.  1.  1.  1.  1.]
```

```
[6]: v4 = np.zeros(10)   #un vecteur nul de dimension 10

print(v4)
```

```
[0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
```

Pour obtenir un vecteur d'images de valeurs d'un vecteur par une fonction, il suffit de donner ce vecteur comme argument à cette fonction.

Exemple

```
[7]: def inverse(x):
    return 1/x

vecteur = np.array([1,2,8])
```

```
print(inverse(vecteur))
```

```
[1.    0.5    0.125]
```

2 Le sous module matplotlib.pyplot

matplotlib.pyplot est une bibliothèque *Python* pour créer des visualisations de données sous forme de graphiques comme les courbes, les nuages de points, les diagrammes en bâtons, les histogrammes, ...

On commence par l'importer sous l'alias *plt* :

```
[8]: import matplotlib.pyplot as plt
```

2.1 Courbe d'une fonction

Après avoir créé les données (vecteur d'abscisses et vecteur d'ordonnées), on fait appel à la fonction *plot(abscisses, ordonnées)* qui permet de tracer une ligne brisée continue entre les points donnés.

Exemple

Tracement de la courbe de $f(x) = \frac{1}{x}$ sur $[1, 10]$:

```
[9]: # définition de la fonction
def f(x):
    return 1/x

# création des abscisses
x = np.linspace(1, 10, 50)    # 50 valeurs sur [1, 10]

# création des ordonnées
y = f(x)

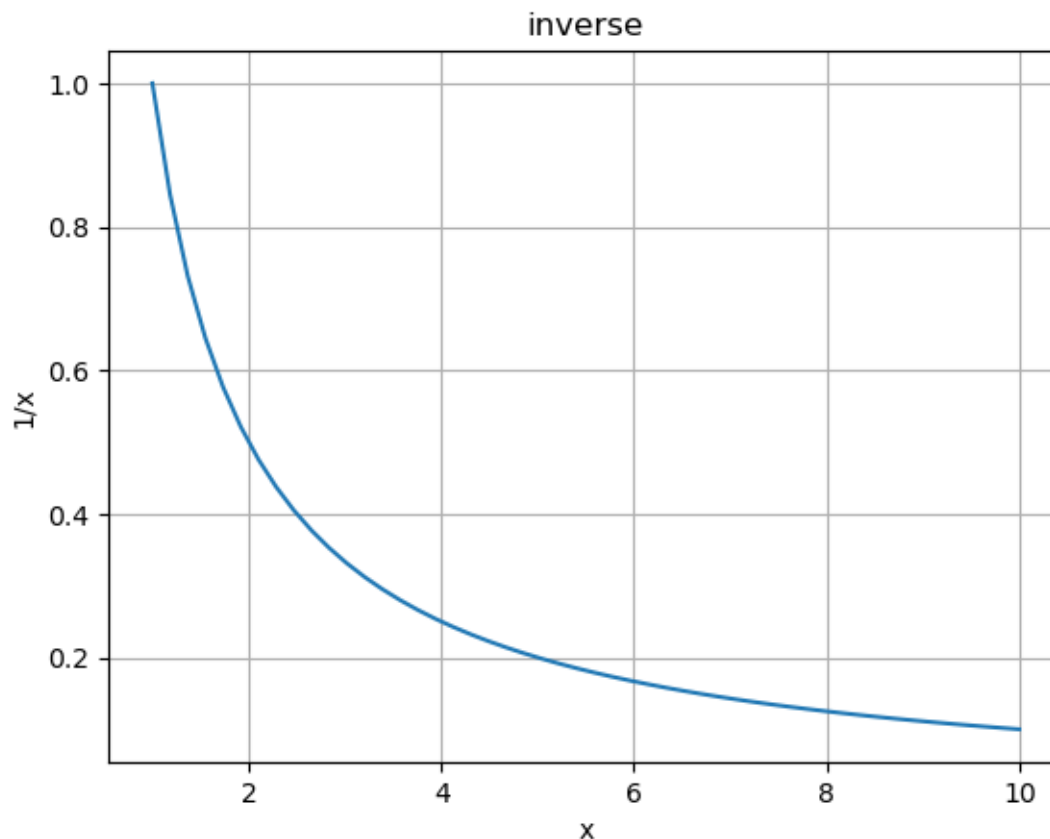
# traçage de la courbe
plt.plot(x, y)

# ajout d'une grille
plt.grid()

# ajout d'une légende aux axes
plt.xlabel('x')
plt.ylabel('1/x')

# ajout d'un titre
plt.title('inverse')

# affichage
plt.show()
```



REMARQUE : Plus le nombre des abscisses est grand, plus la courbe est précise et approchera mieux la forme mathématique de la fonction.

Pour tracer plusieurs courbes sur la même figure il suffit d'ajouter autant de plot que nécessaire et de personnaliser de préférence chaque courbe par :

- un label en utilisant le paramètre *label* pour l'identifier dans la légende;
- une couleur avec le paramètre *color* et des noms de couleurs en anglais;
- un style de ligne avec le paramètre *linestyle* :
 - '-' pour une ligne continue (valeur par défaut)
 - '- -' pour une ligne discontinue
 - '-.' pour une alternance de points et tirets
 - ':' pour une ligne pointillée

Exemple

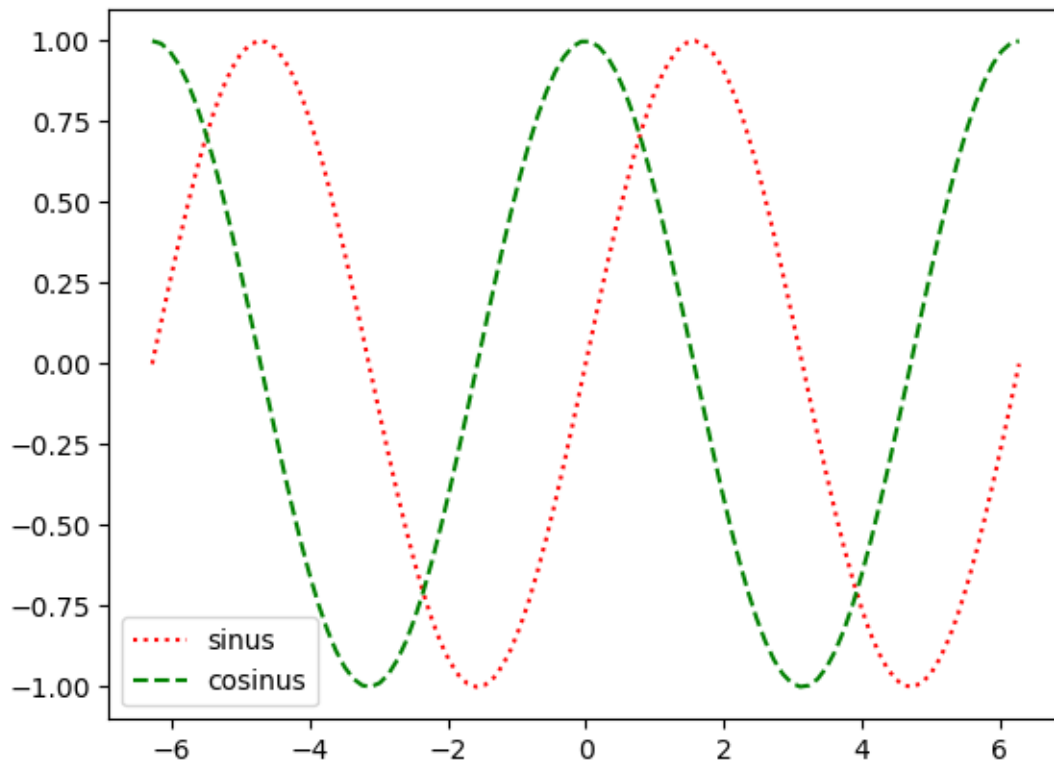
Tracement des courbes des fonctions *sinus* et *cosinus* sur $[-2\pi, 2\pi]$:

```
[10]: x = np.linspace(-2*np.pi, 2*np.pi, 100)
      y1 = np.sin(x)
      y2 = np.cos(x)
```

```
plt.plot(x,y1, label='sinus', color='red', linestyle=':')
plt.plot(x,y2, label='cosinus', color='green', linestyle='--')

plt.legend()    #pour ajouter une légende avec les noms des courbes

plt.show()
```



Pour afficher chaque courbe séparément, il suffit de créer une figure avec la fonction `figure()` et, avant le tracé de chaque courbe, on fait appel à la fonction `subplot` qui permet d'afficher plusieurs graphiques dans une même figure organisés en grille. Pour cela, elle prend comme paramètres le nombre de lignes, de colonnes et l'emplacement de la courbe dans la grille.

```
[11]: x = np.linspace(-2*np.pi, 2*np.pi, 100)

#Figure
plt.figure()

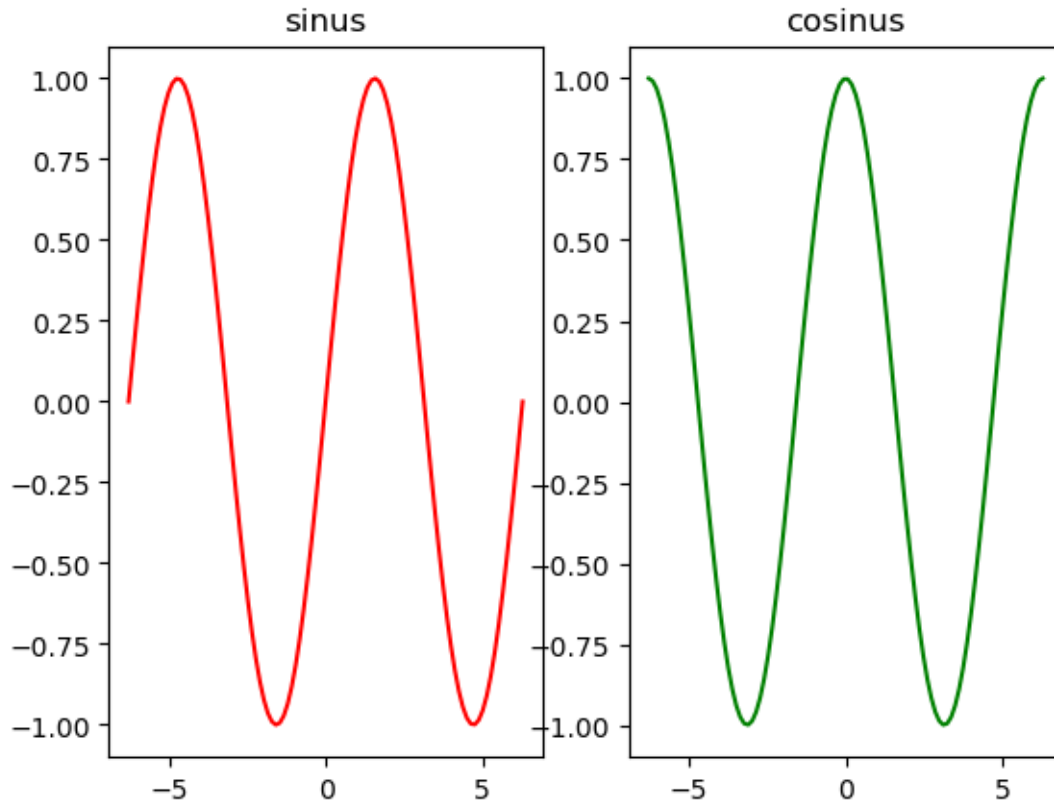
# Affichage de deux courbes en utilisant une seule ligne et deux colonnes

# sinus
plt.subplot(1,2,1)
plt.plot(x, np.sin(x), color='red')
```

```
plt.title('sinus')

# cosinus
plt.subplot(1,2,2)
plt.plot(x, np.cos(x), color='green')
plt.title('cosinus')

plt.show()
```



On peut même ajouter des annotations sur la figure avec la fonction *annotate* en précisant le text à écrire et ses coordonnées :

Exemple

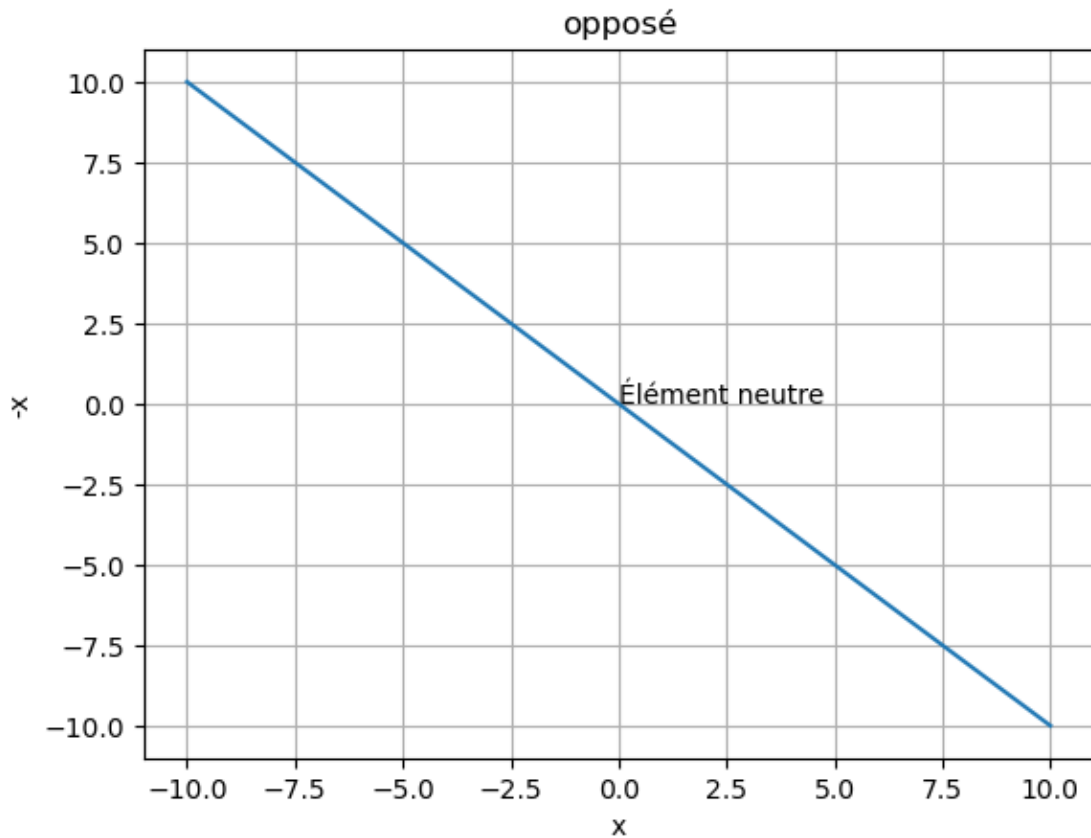
```
[12]: def opposé(x):
      return -x

x = np.arange(-10, 11, 1)
plt.plot(x, opposé(x))

plt.annotate('Élément neutre', (0, 0)) # écrire 'Élément neutre' aux
↳ coordonnées (0, 0)
```

```
plt.xlabel('x')
plt.ylabel('-x')
plt.grid()
plt.title('opposé')

plt.show()
```



3 TP

Exercice 1

Soit la fonction définie par : $\forall x \in \mathbb{R}, f(x) = \frac{e^x - e^{-x}}{2}$.

1. Créer un vecteur X qui contient 100 abscisses uniformément distribuées sur l'intervalle $[-3, 3]$.
2. Créer un autre vecteur qui contient l'image de X par f .
3. Tracer la courbe de f sur $[-3, 3]$.
4. Tracer sans faire de calcul f^{-1} la fonction réciproque de f .

```
[13]: import numpy as np
import matplotlib.pyplot as plt
```

```
[14]: # 1

X = np.linspace(-3, 3, 100)

# 2

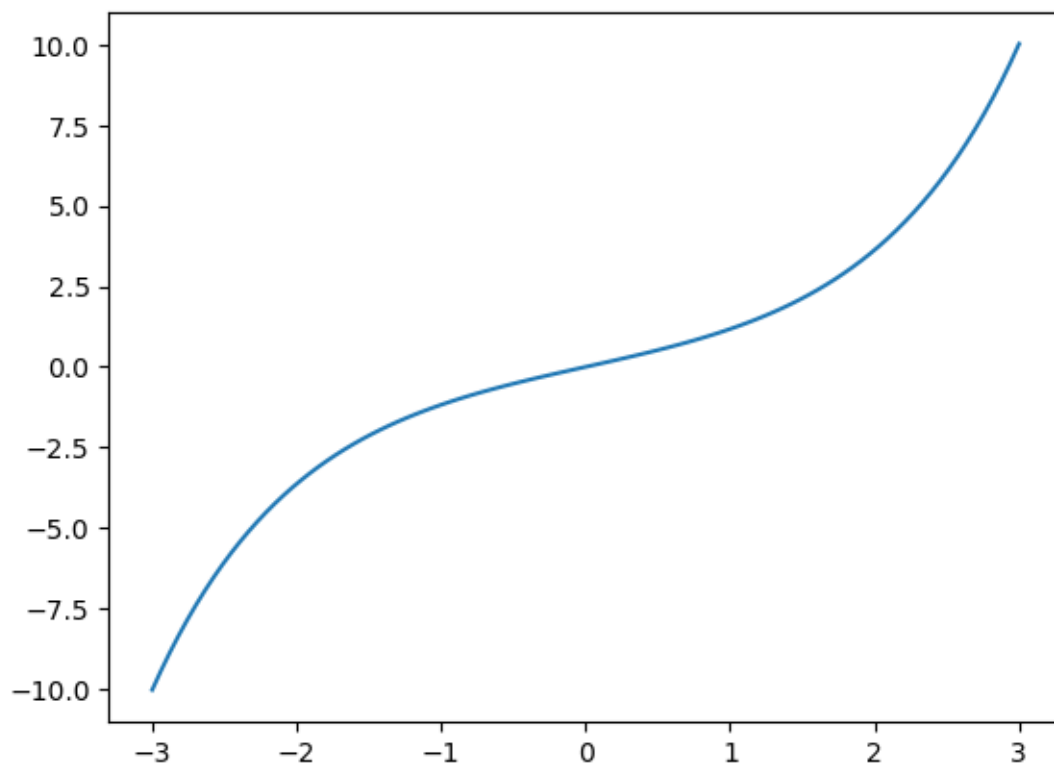
def f(x):
    return (np.exp(x)-np.exp(-x))/2

Y = f(X)

# 3

plt.plot(X, Y)

plt.show()
```



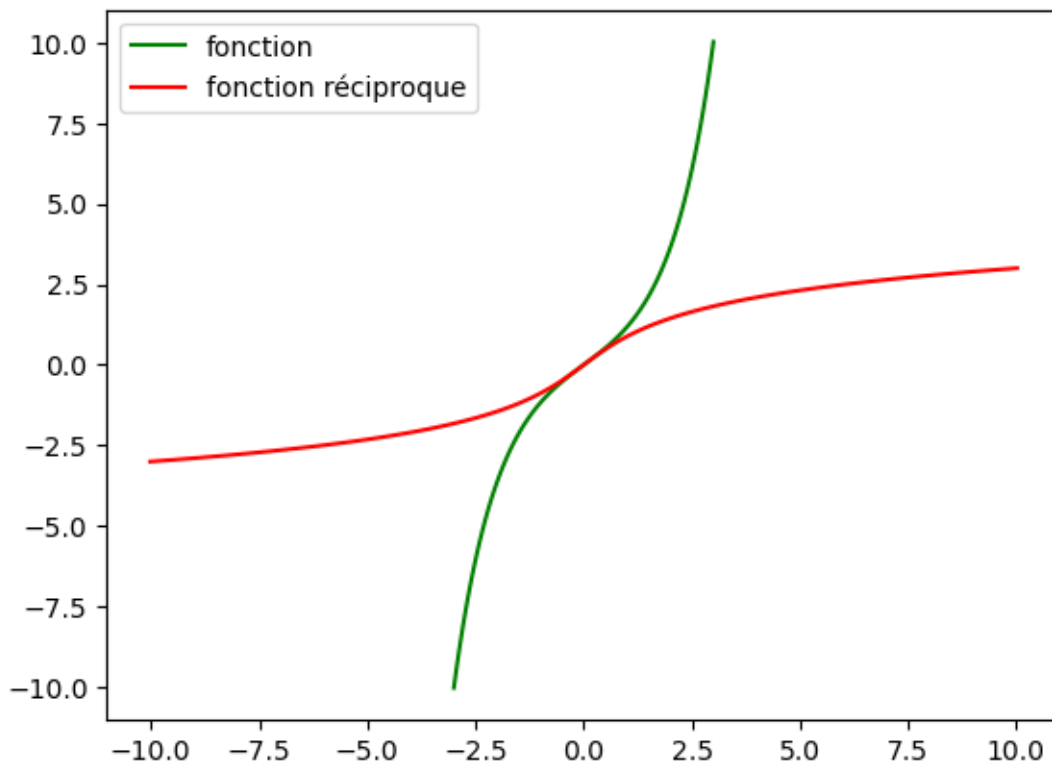
```
[15]: # 4
```

```
plt.plot(X, Y, label='fonction', color='green')

# Les courbes d'une fonction et de sa fonction réciproque sont symétriques par
# rapport à la droite identité,
# il suffit donc d'échanger les axes :

plt.plot(Y, X, label='fonction réciproque', color='red')

plt.legend()
plt.show()
```



Exercice 2

Soit la fonction définie par : $\forall x \in \mathbb{R}, f(x) = x - \ln(1 + |x|)$

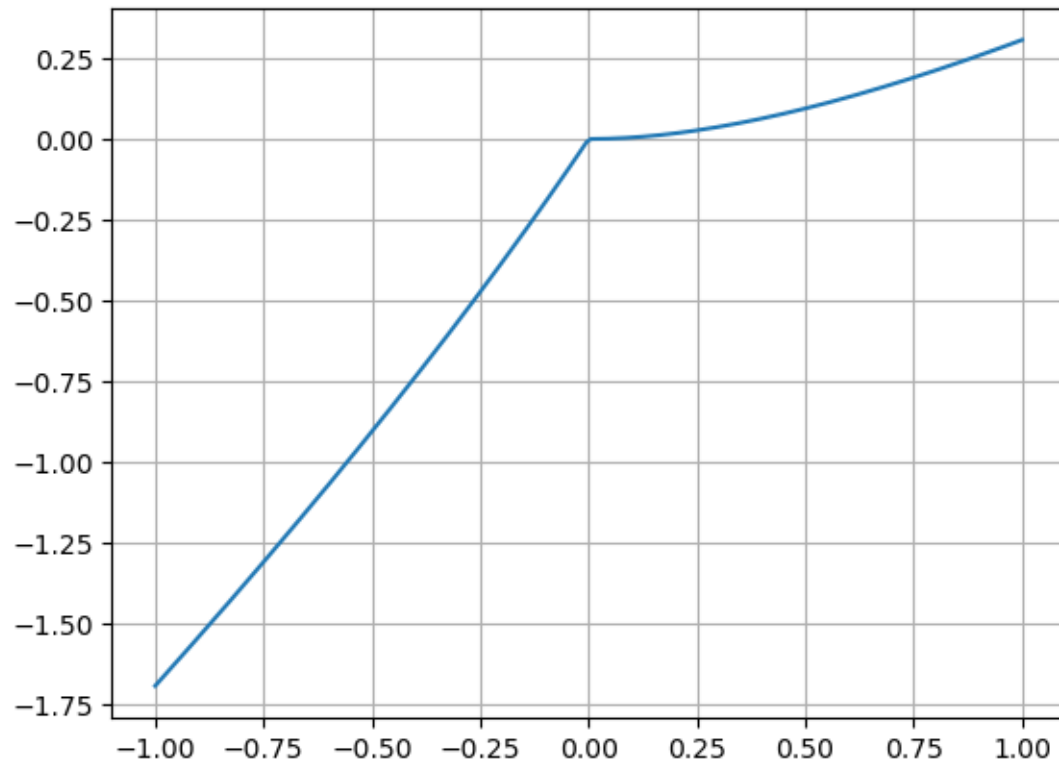
1. Tracer le graphe de f sur l'intervalle $[-1, 1]$ dans un repère muni d'une grille.
2. Dédurre graphiquement la dérivabilité de f en 0.

```
[16]: # 1

X = np.linspace(-1, 1, 200)

def f(x):
```

```
return x-np.log(1+np.absolute(x))  
  
plt.plot(X, f(X))  
plt.grid()  
plt.show()
```



2. Le graphe présente deux tangentes différentes à gauche et à droite en 0, ce qui veut dire que $f'_-(0) \neq f'_+(0)$, alors la fonction n'est pas dérivable en 0.